Análisis y Diseño de Software

Autores

Laínez Suárez Ricardo Andrés

Padilla Triviño Marco Adrian

Andino Martínez Nelson David

NRC: 36838

Planificación y trazabilidad del sistema Bocatto

Informe de historias de usuario, sprints y trazabilidad

Ing. Jenny Ruiz

Departamento de Ciencias de la Computación - ESPE

Índice

1. Introducción **3**

2. Desarrollo **3**

2.1. Historias de usuario (matriz MTZHU) 3

2.2. Backlog y sprints 4

2.3. Jira 4

2.4. Confluence 4

2.5. Trazabilidad con prompt y diagrama de clases 5

2.6. Dónde está cada evidencia 6

3. Conclusiones **6**

1. Introducción

Este informe resume cómo el Grupo 5 planificó el sistema de gestión de pedidos Bocatto y cómo se mantiene la trazabilidad de cada requisito. La planificación toma los 4 casos de uso de nivel 0 que se trabajaron en la Guía 1 y la Guía 2: tomar pedido en mesa, realizar pedido por QR, gestionar la preparación y cobrar en caja. Las guías del repositorio sí incluyen los 8 casos de uso.

Objetivo: documentar la trazabilidad de los requisitos RF-02, RF-04, RF-05 y RF-06 desde la ERS hasta su historia de usuario, sus tareas en Jira, su pantalla en Confluence y su diagrama de clases generado con un prompt de ingeniería, indicando dónde se encuentra cada evidencia.

2. Desarrollo

2.1. Historias de usuario (matriz MTZHU)

Las historias se llenaron en la plantilla de la asignatura `MTZHU_G5_V1.xlsx`. La hoja *Formato descripción HU* recoge, por cada ítem, el problema, la necesidad, la solución, el usuario, las tareas, el responsable, las horas, la fecha de entrega, la prioridad, el estado, la prueba de aceptación y los comentarios. Cada historia tiene además su ficha en una hoja aparte (Figura 1).

Matriz de Marco de Trabajo de HU Sistema de Gestión de Pedidos Bocatto

ITEM	PROBLEMA	QUE (NECESIDAD)	PARA QUE (SOLUCIÓN)	PARA QUIEN (USUARIO)	COMO (DESCRIPCIÓN DE TAREAS)	HECHO POR (PROG. RESP.)	TIEMPO (ESTIMADO EN HRS)	FECHA DE ENTREGA	PRIORIDAD	STATUS	PRUEBA (COMO SE VERIFICA)	COMENTARIOS	NOMBRE DE HISTORIA
REQ001	Los pedidos en mesa se anotan a mano, lo que provoca errores al registrar las órdenes y demoras al llevarlas a cocina.	Registrar el pedido de una mesa numerada con productos, cantidades y observaciones.	Enviar el pedido a cocina sin errores de anotación.	Mesero	1. Diseñar la pantalla de toma de pedido con selección de mesa. (Marco Padilla, 1 h) 2. Implementar el registro de productos, cantidades y observaciones. (Marco Padilla, 2 h) 3. Validar que el pedido tenga al menos un producto y generar su identificador. (Nelson Andino, 1 h) 4. Enviar el pedido en estado pendiente a la pantalla de cocina. (Ricardo Lainez, 2 h)	Marco Padilla	6	2026-11-08	Alta	Terminado	Dado que el mesero selecciona una mesa y agrega al menos un producto, cuando confirma el pedido, entonces el sistema genera su identificador, lo deja en estado pendiente y lo muestra en la pantalla de cocina.	Caso de uso CU 0-1. Debe tener al menos un producto. Se genera un identificador y el pedido inicia en estado pendiente.	Tomar pedido en mesa
REQ004	El cliente depende del mesero para hacer su pedido, lo que genera esperas y demoras en la atención.	Escanear el QR de la mesa y elegir productos del menú digital.	Hacer el pedido desde el celular sin esperar al mesero.	Cliente	1. Diseñar el menú digital para celular que se abre desde el QR. (Marco Padilla, 2 h) 2. Implementar la identificación de la mesa a partir del código QR. (Ricardo Lainez, 2 h) 3. Registrar el pedido del cliente en estado pendiente y enviarlo a cocina. (Ricardo Lainez, 2 h)	Marco Padilla	6	2026-11-09	Alta	Terminado	Dado que el cliente escanea el QR de su mesa, cuando elige productos del menú digital y confirma, entonces el sistema registra el pedido de esa mesa en estado pendiente y lo envía a cocina.	Caso de uso CU 0-3. El sistema identifica la mesa por el QR y muestra precios y promociones vigentes.	Realizar pedido por QR
REQ005	En cocina no hay claridad sobre el orden y el estado de cada pedido, y el personal no sabe cuándo está listo.	Ver los pedidos pendientes y actualizar su estado.	Que el personal y el cliente sepan en qué va cada pedido.	Cocinero	1. Diseñar la pantalla de cocina con la lista de pedidos pendientes. (Ricardo Lainez, 1 h) 2. Implementar el cambio de estado en secuencia válida. (Marco Padilla, 2 h) 3. Reflejar el estado actualizado para el personal y el cliente. (Nelson Andino, 2 h)	Ricardo Lainez	5	2026-11-10	Alta	Terminado	Dado que hay un pedido pendiente, cuando el cocinero cambia su estado, entonces el sistema solo acepta la secuencia pendiente, en preparación, listo y listo para entregar, y muestra el nuevo estado.	Caso de uso CU 0-4. Secuencia de estados: pendiente, en preparación, listo y listo para entregar.	Gestionar preparación del pedido
REQ006	En caja cuesta ubicar el pedido de cada mesa y calcular el total a cobrar sin errores.	Buscar el pedido por número de mesa y registrar el método de pago.	Cobrar el total correcto y habilitar el pedido para la entrega.	Cajero	1. Diseñar la pantalla de caja con búsqueda por número de mesa. (Nelson Andino, 1 h) 2. Mostrar el detalle del pedido y calcular el total a pagar. (Marco Padilla, 2 h) 3. Registrar el método de pago y marcar el pedido como pagado. (Nelson Andino, 2 h)	Nelson Andino	5	2026-11-12	Alta	Terminado	Dado que el pedido de una mesa está listo, cuando el cajero la busca por número de mesa y registra el método de pago, entonces el sistema muestra el total y marca el pedido como pagado.	Caso de uso CU 0-5. Solo se cobra un pedido listo. Pago en efectivo o digital/transferencia.	Cobrar pedido en la caja

Figura 1: Matriz de marco de trabajo de historias de usuario con los 4 requisitos.

2.2. Backlog y sprints

El backlog (`G5_Backlog_PlantillaV1.xlsx`) tiene una historia por requisito con sus tareas `REQ00X-N`. Se definió un sprint por caso de uso. Los cuatro están cerrados y el último terminó el 12 de noviembre a las 09:00.

Sprint	Fecha	Requisito	Caso de uso	Encargado	Horas
Sprint 1	8 nov	RF002 Tomar pedido en mesa	CU 0-1	Marco Padilla	6
Sprint 2	9 nov	RF004 Realizar pedido por QR	CU 0-3	Marco Padilla	6
Sprint 3	10 nov	RF005 Gestionar preparación	CU 0-4	Ricardo Laínez	5
Sprint 4	11 al 12 nov	RF006 Cobrar pedido en caja	CU 0-5	Nelson Andino	5

Tabla 1: Sprints, requisitos y encargados. Las subtareas suman 9 h para Marco Padilla, 7 h para Ricardo Laínez y 6 h para Nelson Andino.

2.3. Jira

En el proyecto Scrum **36838_G5** cada sprint es una épica que agrupa su historia. Cada historia lleva el requisito en el nombre y, en la descripción, la historia de usuario, el problema, el caso de uso, la prueba de aceptación, las tareas y la trazabilidad. Sus subtareas tienen responsable y horas, y enlaza a Confluence, al archivo de respaldo y al diagrama de clases.

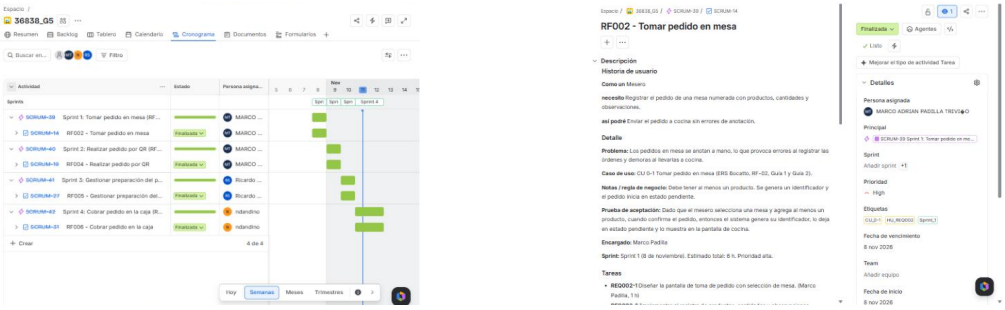


Figura 2: Cronograma de Jira con los 4 sprints (izquierda) e historia SCRUM-14 con su descripción (derecha).

2.4. Confluence

En el espacio 36838_G5, la página *Maquetado del sistema Bocatto por requisito* tiene una subpágina por requisito y otra con el modelo de datos. Cada subpágina muestra la historia de usuario, el sprint y el encargado, el enlace a Jira, la trazabilidad, la prueba de aceptación,

el mockup de la pantalla, el diagrama de clases, los datos que usa y las reglas de negocio (Figura 3, izquierda).

2.5. Trazabilidad con prompt y diagrama de clases

Para cada requisito se redactó un requisito atómico y se usó un modelo de lenguaje con un prompt de ingeniería para generar el código PlantUML de su diagrama de clases. El prompt original se incorporó dentro del código como comentario de metadatos con las etiquetas `/' ... '/`. Así, el modelo queda ligado al requisito, al caso de uso, a la historia y a la tarea de Jira. El fragmento siguiente corresponde a REQ002 y su diagrama está en la Figura 3, a la derecha.

```
' REQ002_TomarPedidoMesa.txt
@startuml
/'
METADATOS DE TRAZABILIDAD
Requisito: REQ002 (RF-02) | Caso de uso: CU 0-1 | Jira: SCRUM-14 (Sprint 1)
Prompt: Actúa como un Analista de Software y Diseñador UML. Con base en el
    siguiente requisito atómico, genera el código PlantUML de un Diagrama de
    Clases que modele la ontología básica de esta funcionalidad (frontera,
    controlador, entidades y persistencia desacoplada)...
Requisito Atómico: "El sistema Bocatto deberá permitir al Mesero registrar el
    pedido de una mesa numerada con sus productos, cantidades y observaciones..."
'/
```

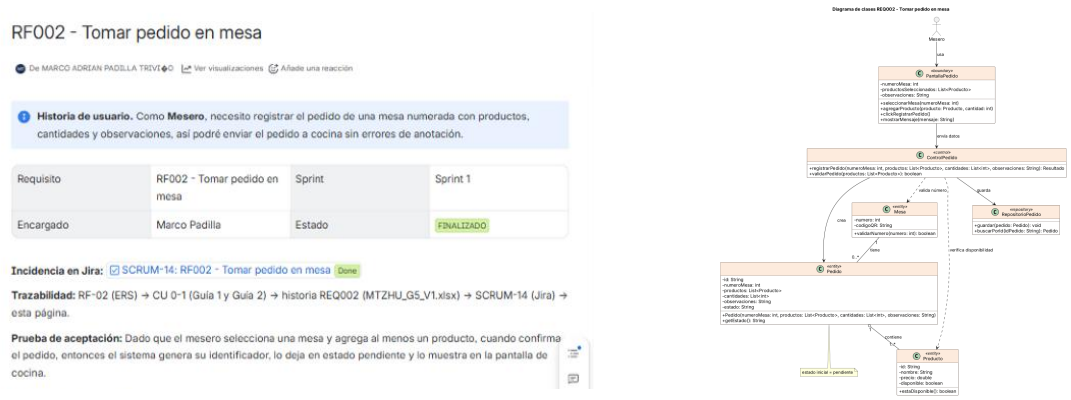


Figura 3: Página de Confluence de RF002 con su trazabilidad (izquierda) y diagrama de clases de REQ002 generado a partir del prompt (derecha).

La ficha de trazabilidad de cada requisito está en el README de `1.8 Archivos de Respaldo`.

2.6. Dónde está cada evidencia

El repositorio se organizó en la carpeta `PROYECTO`, con `1. ELICITACION`, `2. DISEÑOS` y `3. CODIGO`. En Notion, la sección *SPRINTS - JIRA* del espacio P.A.R.A. tiene un enlace por sprint, y la pestaña *Archivos* guarda cada documento con su archivo original y su enlace a GitHub (Figura 4).

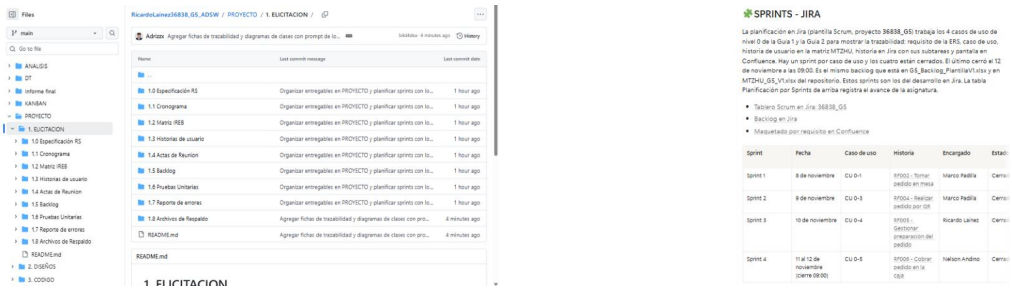


Figura 4: Carpeta de elicitación en GitHub (izquierda) y sección SPRINTS - JIRA en Notion (derecha).

Tabla 2: Ubicación de cada evidencia.

Evidencia	**Ubicación**
Historias de usuario	PROYECTO/1. ELICITACION/1.3 Historias de usuario
Backlog y cronograma	1.5 Backlog y 1.1 Cronograma
Fichas de trazabilidad	PROYECTO/1. ELICITACION/1.8 Archivos de Respaldo
Diagramas de clases	PROYECTO/2. DISEÑOS/2.5 Diagramas de clases por requisito
Guías, mockups y modelo	PROYECTO/2. DISEÑOS (2.1, 2.2 y 2.3 Mockups)
Jira (tablero)	36838-g5.atlassian.net/jira/software/projects/SCRUM/boards/1
Confluence	`https://36838-g5.atlassian.net/wiki/spaces/3/pages/327682`
Notion (P.A.R.A.)	`https://www.notion.so/3f3cd8194c458048b3adf5c09e2195b6`
NotebookLM	Cuaderno 1: requisitos y Cuaderno 2: modelado UML
Gem Tutor Bocatto	`https://gemini.google.com/gem/d9eeb390f7c5?usp=sharing`

3. Conclusiones

Cada requisito se sigue sin saltos desde la ERS hasta su historia, sus tareas en Jira, su pantalla y su diagrama de clases. Guardar el prompt dentro del código PlantUML deja registrado cómo se obtuvo cada modelo.